DOCUMENTACIÓN TÉCNICA

Microservicio de Productos — ms-products

1. Resumen

MS-Products es un microservicio REST destinado a administrar un catálogo de productos de una plataforma e-commerce. El proyecto utiliza Java 17, Spring Boot, Spring MVC, Spring Data JPA, Spring Security y MySQL. Esta documentación distingue lo implementado en código de lo descrito como objetivo en README y scripts SQL.

2. Alcance y estado funcional

Función	Estado observado	Evidencia
Listar productos activos	Implementada	GET /api/products; findByActiveTrue()
Consultar por ID	Implementada	GET /api/products/{id}
Registrar producto	Implementada	POST /api/products
Modificar stock	Implementada	PATCH /api/products/{id}/stock
Dar de baja producto	Implementada (lógica)	DELETE /api/products/{id}; active=false
Actualizar producto completo	Pendiente	updateProduct() devuelve null; no hay @PutMapping

3. Tecnologías y dependencias

Componente	Uso
Java 17	Lenguaje / nivel configurado en pom.xml
Spring Boot 4.1.1	Parent de Maven
Spring Web MVC	Controladores HTTP y respuestas JSON
Spring Data JPA	Persistencia con JpaRepository
MySQL Connector/J	Conexión a MySQL
Spring Security	HTTP Basic, roles y @PreAuthorize
JUnit / Mockito	Pruebas unitarias del servicio
JWT	Dependencias incluidas; no se identificó uso activo de JWT

4. Arquitectura y recorrido de una solicitud

Cliente HTTP (Postman / otro servicio) → ProductController → IProductService / ProductServiceImpl → ProductRepository (JPA) → MySQL. ProductServiceImpl convierte las entidades Product a ProductResponseDTO antes de devolver la respuesta al controlador. SecurityConfig aplica filtros HTTP y las anotaciones @PreAuthorize evalúan los roles en cada método.

Paquete/clase	Responsabilidad
controller/ProductController	Declara rutas REST y delega operaciones
service/IProductService	Contrato de operaciones
service/ProductServiceImpl	Implementación de reglas de negocio

repository/ProductRepository	Acceso a datos; consulta findByActiveTrue
model/Product	Entidad persistente y atributos
dto/*	Objetos de entrada y salida
security/SecurityConfig	Autenticación y autorización
exceptions/GlobalExceptionHandler	Convierte excepciones de dominio en respuestas HTTP

5. API REST

Ruta base: /api/products.

Método y ruta	Función	Rol exigido por @PreAuthorize
GET /api/products	Lista productos activos	CLIENT o ADMIN
GET /api/products/{id}	Consulta por identificador	CLIENT o ADMIN
POST /api/products	Crea producto	CLIENT o ADMIN (método); ADMIN (filtro HTTP)
PATCH /api/products/{id}/stock?stock=N	Actualiza unidades	ADMIN
DELETE /api/products/{id}	Baja lógica	ADMIN

5.1. Registro de producto

Ejemplo orientativo de cuerpo JSON: {"name":"Laptop","description":"Equipo portátil","price":25000.00,"stock":7}. ProductRequestDTO incluye name, description, price y stock. El controlador devuelve ResponseEntity.ok(), equivalente a HTTP 200 cuando la operación finaliza normalmente; no devuelve 201 Created.

5.2. Actualización de stock

PATCH /api/products/1/stock?stock=5; el valor stock se recibe mediante @RequestParam. Si es nulo o negativo se lanza IllegalArgumentException, para la cual no hay un manejador específico en GlobalExceptionHandler.

5.3. Borrado lógico

DELETE /api/products/1 no elimina físicamente la fila: establece active=false y guarda la entidad. Si el ID no existe se devuelve 404; si ya estaba inactivo se devuelve 409.

6. Modelo de datos

Campo Product	Tipo Java / particularidad
id	Integer; clave primaria autogenerada
name	String; longitud 150, obligatorio
description	String; TEXT
price	BigDecimal; precisión 10, escala 2
stock	Integer; obligatorio
active	Boolean; valor inicial true
createdAt	LocalDateTime; columna created_at

El archivo db/01_create_tables.sql define el esquema ecommerce y tablas categories y products; products incluye category_id como clave foránea. Sin embargo, la entidad Product actual NO mapea category_id ni define una relación con Category. La configuración application.properties apunta a products_MS y spring.jpa.hibernate.ddl-auto=validate. Deben conciliarse los esquemas antes de validar el arranque.

7. Seguridad

- Se habilita seguridad a nivel de método mediante @EnableMethodSecurity y @PreAuthorize.
- Se configura autenticación HTTP Basic con usuarios definidos en memoria para desarrollo.
- El filtro HTTP restringe POST, PUT y DELETE a ADMIN; el método de POST declara CLIENT o ADMIN, pero prevalece la restricción previa de filtro para POST.
- La ruta PATCH exige ADMIN en el método; GET exige CLIENT o ADMIN en el método.
- CSRF está deshabilitado. Las contraseñas de ejemplo aparecen en el archivo fuente: deben retirarse de cualquier entrega de producción y sustituirse por gestión segura de identidades.

8. Manejo de errores

Excepción	Estado HTTP manejado
ProductNotFoundException	404 Not Found
NoProductsAvailableException	404 Not Found
ProductAlreadyInactiveException	409 Conflict
IllegalArgumentException (stock inválido)	Sin manejador específico documentado

El cuerpo de error manejado tiene forma {"message":"descripción"}. El comportamiento de otros errores depende de los mecanismos predeterminados de Spring Boot y no se verificó mediante ejecución.

9. Instalación y ejecución local

- Requisitos: JDK 17, una instancia MySQL disponible y configuración de usuario/contraseña válida.
- Alinear primero la base indicada en application.properties con las tablas realmente existentes y el modelo JPA.
- En la raíz del proyecto: .\mvnw.cmd spring-boot:run (Windows).
- Probar los endpoints usando un cliente HTTP con autenticación Basic y usuarios/roles de desarrollo válidos.
- No se incluye en este documento ninguna credencial extraída del archivo recibido.

10. Pruebas y verificación

El proyecto incluye MsProductsApplicationTests y ProductServiceImplTest (JUnit 5 / Mockito). Se identificaron pruebas unitarias del servicio; no se ejecutaron en este análisis. Comando recomendado en Windows: .\mvnw.cmd test. Adjuntar resultados reales o capturas de Postman antes de aprobar la documentación como evidencia de funcionamiento.